

What's new in

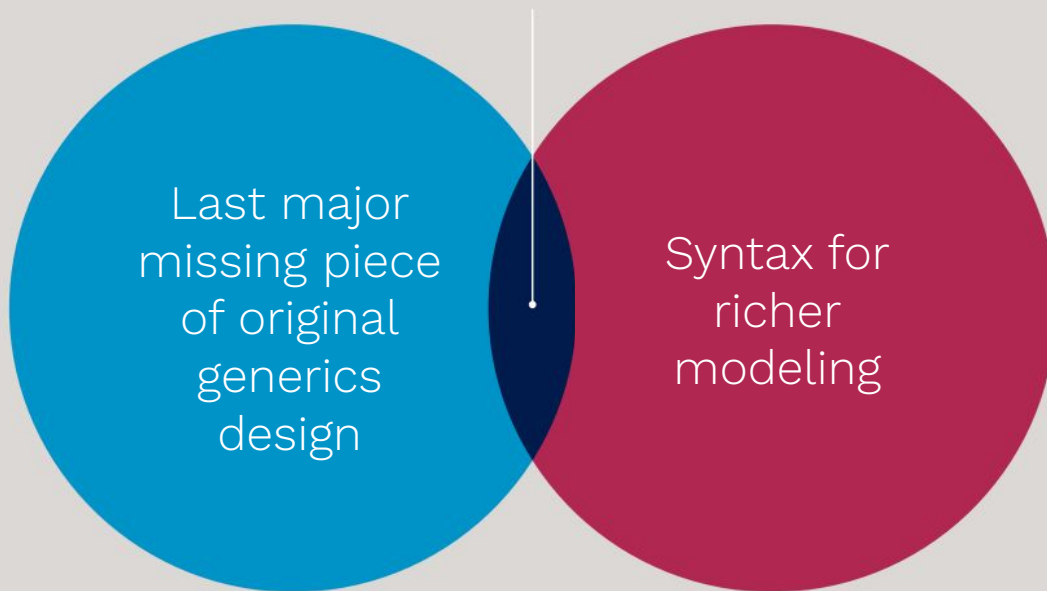


// Who am I?

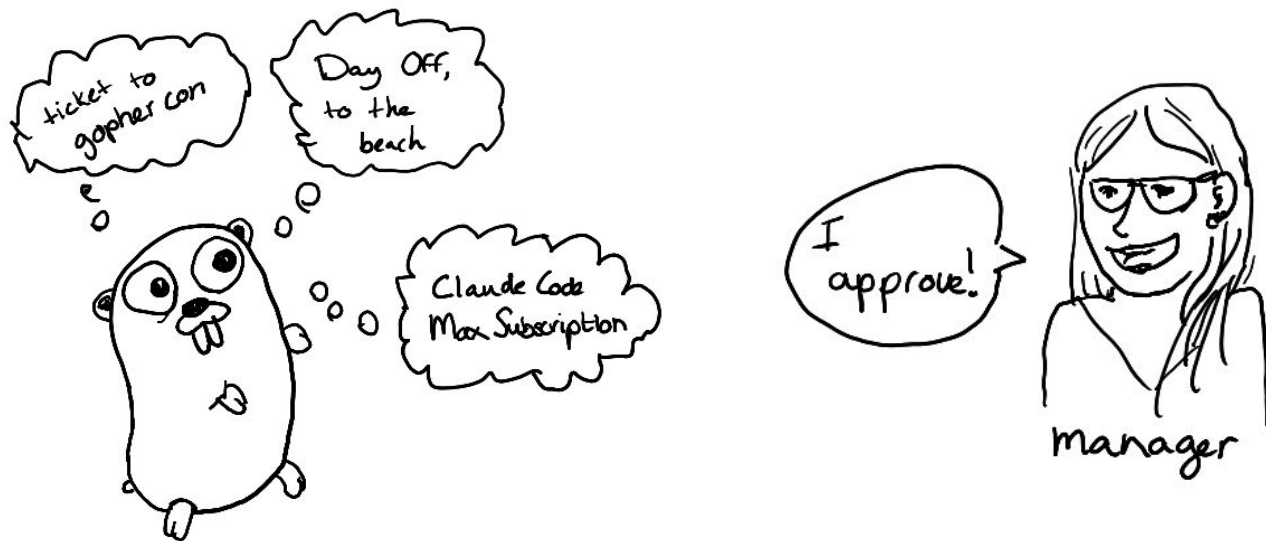
Loes Alleman

- 12 years in software engineering at Visma | ProActive
- New to Go
 - PHP background, mostly
 - Adopting Go since a few months, but AI does all the typing, so...
- Linguistic background
- Focus on “Socio-Technical” architecture
- Enabling AI adoption (organisation-wide)

New in 1.27: generic methods



Our system: approving employee requests



Golang 1.27: generic methods

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

// Go 1.27: The struct method has a type parameter [T Request].
func (m Manager) Approve[T Request](req T) T {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    // The compiler knows 'dayOff' is a *DayOff.
    dayOff := manager.Approve(&DayOff{Date: "Friday"})
    fmt.Println("Is Friday approved?", dayOff.Approved)
}
```

Golang 1.27: generic methods

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

// Go 1.27: The struct method has a type parameter [T Request].
func (m Manager) Approve[T Request](req T) T {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    // The compiler knows 'dayOff' is a *DayOff.
    dayOff := manager.Approve(&DayOff{Date: "Friday"})
    fmt.Println("Is Friday approved?", dayOff.Approved)
}
```

Golang 1.18: generics / type parameters

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

// Go 1.18: Standalone function needed.
func Approve[T Request](m Manager, req T) T {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    // The manager goes in the generic function.
    // But the return type is still strictly typed, fortunately.
    ticket := Approve(manager, &ConferenceTicket{Event: "GopherCon"})
    fmt.Println("Is GopherCon approved?", ticket.Approved)
}
```

Golang 1.18: generics / type parameters

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

// Go 1.18: Standalone function needed.
func Approve[T Request](m Manager, req T) T {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    // The manager goes in the generic function.
    // But the return type is still strictly typed, fortunately.
    ticket := Approve(manager, &ConferenceTicket{Event: "GopherCon"})
    fmt.Println("Is GopherCon approved?", ticket.Approved)
}
```

Golang 1.17: describing behavior omits type

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

// Go 1.17: Accepts an interface, returns an interface.
// The compiler doesn't know the type.
func (m Manager) Approve(req Request) Request {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    subscription := manager.Approve(&Subscription{Name: "ClaudeCode Max"})

    // The compiler needs the pointer to the struct to access it
    sub := subscription.(*Subscription)
    fmt.Println("Is the license approved?", sub.Approved)
}
```

Golang 1.17: describing behavior omits type

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

// Go 1.17: Accepts an interface, returns an interface.
// The compiler doesn't know the type.
func (m Manager) Approve(req Request) Request {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    subscription := manager.Approve(&Subscription{Name: "ClaudeCode Max"})

    // The compiler needs the pointer to the struct to access it
    sub := subscription.(*Subscription)
    fmt.Println("Is the license approved?", sub.Approved)
}
```

Why it matters?



Golang 1.27: generic methods

```
package main

import "fmt"

type Request interface {
    SetApproved()
}

type DayOff struct { Date string; Approved bool }
func (d *DayOff) SetApproved() { d.Approved = true }

type Subscription struct { Name string; Approved bool }
func (s *Subscription) SetApproved() { s.Approved = true }

type ConferenceTicket struct { Event string; Approved bool }
func (c *ConferenceTicket) SetApproved() { c.Approved = true }

type Manager struct { Name string }

func (m Manager) Approve[T Request](req T) T {
    req.SetApproved()
    fmt.Printf("%s approved a request.\n", m.Name)
    return req
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    dayOff := manager.Approve(&DayOff{Date: "Friday"})
    fmt.Println("Is Friday approved?", dayOff.Approved)
}
```

Syntax allows for a richer model

```
package main
import (
    "errors"
    "fmt"
)

type Request interface {
    SetApproved() error
}

type DayOff struct { Date string; DaysRemaining int; Approved bool }
func (d *DayOff) SetApproved() error {
    if d.DaysRemaining <= 0 {
        return errors.New("insufficient leave balance")
    }
    d.Approved = true
    return nil
}

type ConferenceTicket struct { Event string; Cost int; Approved bool }
func (c *ConferenceTicket) SetApproved() error {
    if c.Cost > 500 {
        return errors.New("exceeds departmental budget")
    }
    c.Approved = true
    return nil
}

type Manager struct { Name string }
func (m Manager) Process[T Request](req T) (T, error) {
    err := req.SetApproved()
    return req, err
}
```

```
func main() {
    manager := Manager{Name: "Loes"}

    dayOff := &DayOff{Date: "Wednesday", DaysRemaining: 5}
    processedDayOff, _ := manager.Process(dayOff)
    fmt.Println("Day off approved:", processedDayOff.Approved)

    ticket := &ConferenceTicket{Event: "GopherCon", Cost: 600}
    processedTicket, err := manager.Process(ticket)
    fmt.Println("Ticket approved:", processedTicket.Approved,
        "| Reason:", err)
}
```



Syntax allows for a richer model - evolvability

```
package main

import (
    "errors"
    "fmt"
)

type RequestForTeam interface {
    AddApproval(teammate string) error
}

type DayOff struct {Approvers []string, Approved bool}
func (d *DayOff) AddApproval(teammate string) error {
    for _, a := range d.Approvers {
        if a == teammate {
            return errors.New("already approved by this teammate")
        }
    }

    d.Approvers = append(d.Approvers, teammate)
    if len(d.Approvers) >= 2 {
        d.Approved = true
    }
    return nil
}

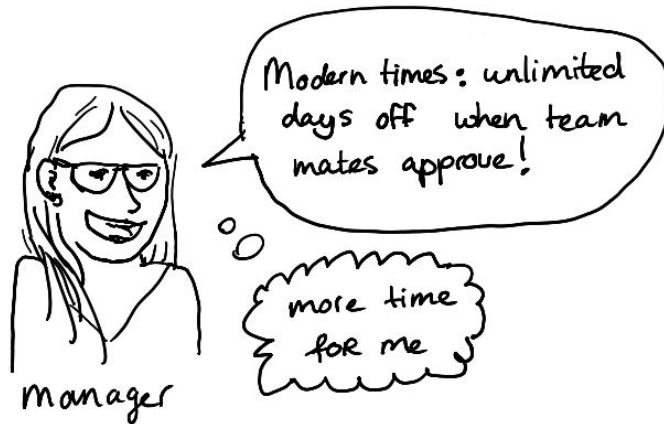
type Teammate struct { Name string }
func (t Teammate) Assess[T RequestForTeam](req T) (T, error) {
    err := req.AddApproval(t.Name)
    if err == nil {
        fmt.Printf("%s assessed a %T.\n", t.Name, req)
    }
    return req, err
}
```

```
func main() {
    teammate1 := Teammate{Name: "Alice"}
    teammate2 := Teammate{Name: "Bob"}

    dayOff := &DayOff{Date: "Wednesday"}

    // First assessment
    teammate1.Assess(dayOff)
    fmt.Println("Is DayOff approved after Alice?", dayOff.Approved)

    // Second assessment triggers the approval
    processedDayOff, _ := teammate2.Assess(dayOff)
    fmt.Println("Is DayOff approved after Bob?", processedDayOff.Approved)
}
```



// the end

thanks for
listening

// tone of voice

Go was created by a team of programmers who value honest, open communication, without exaggeration or excess. Our dialogue with one another and the broader community adheres to the following:

GOPHERS ARE

Concise

Genuine

Friendly

Direct

Thoughtful

Humble

GOPHERS ARE NOT

Verbose

Dubious

Exclusive

Ambiguous

Reactive

Haughty